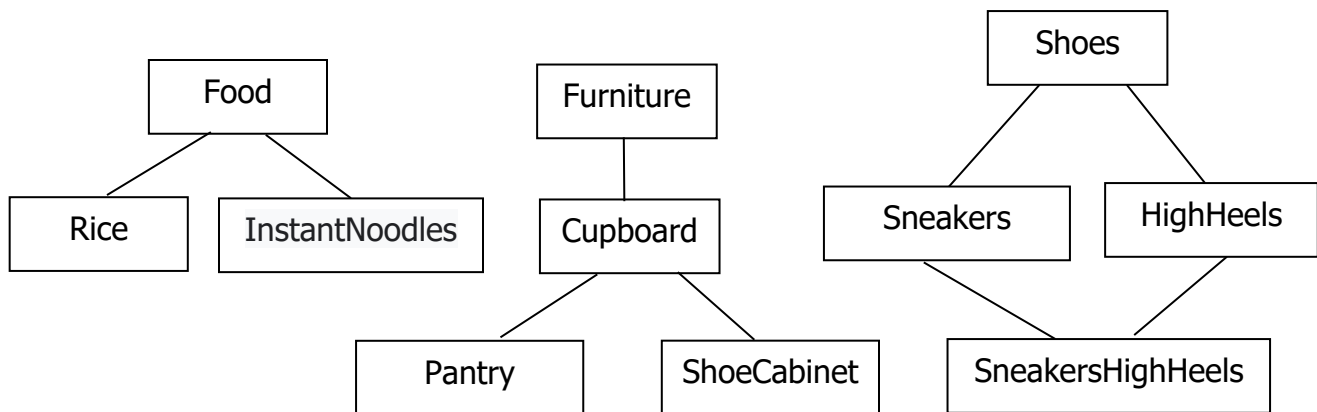


แบบฝึกหัดที่ 59

จงสร้างคลาส Food, Rice, InstantNoodles, Furniture, Cupboard, Pantry, ShoeCabinet, Shoes, Sneakers, HighHeels, SneakersHighHeels โดยมีการรับทอดตามรูปและไม่เกิดความกำกวม การรับทอดทั้งหมดให้ทำให้เป็นการรับทอดแบบ public และกำหนดให้ Furniture, Cupboard เป็น abstract class โดยให้สร้างฟังก์ชัน show เป็น pure virtual function ทุกคลาสให้เก็บข้อมูลทั้งหมดไว้ในส่วนของ private



แต่ละคลาสประกอบด้วยแอตทริบิวต์และความหมาย ดังนี้

คลาส	แอตทริบิวต์	ความหมาย
Food (อาหาร)	double price;	ราคา
Rice (ข้าว)	int type; int weight;	1.ข้าวหอมมะลิ 2.ข้าวไรซ์เบอร์รี่ 3.ข้าวกล้อง 4.ข้าวมันญี่ปุ่น 5.ข้าวสังข์หยดพัทลุง น้ำหนัก เช่น 5 kg, 2 kg, 1 kg
InstantNoodles (บะหมี่กึ่งสำเร็จรูป)	int flavor;	1.ต้มยำกุ้ง 2.หมูสับ 3.หมูมะนาว 4.หมาล่า
Shoes (รองเท้า)	double price; double size;	ราคา ขนาดรองเท้า
Sneakers (รองเท้าผ้าใบ)	bool shoelace;	แบบผูกเชือก หรือ ไม่ผูกเชือก
HighHeels (รองเท้าส้นสูง)	double height;	ความสูงของส้นรองเท้า (ส้นหลัง)
SneakersHighHeels (รองเท้าผ้าใบส้นสูง)	double frontHeight;	ความสูงของส้นหน้า
Furniture (เฟอร์นิเจอร์)	double price;	ราคา
Cupboard (ตู้)	int color;	1.ขาว 2.ดำ 3.น้ำตาล
Pantry (ตู้เก็บข้าว)	int numFood; Food **food;	ขนาดของอาร์เรย์ food อาร์เรย์ขนาด numFood เก็บพอยเตอร์ที่ชี้ไปยัง Food
ShoeCabinet (ตู้เก็บรองเท้า)	int numShoes; Shoes **shoes;	ขนาดของอาร์เรย์ shoes อาร์เรย์ขนาด numShoes เก็บพอยเตอร์ที่ชี้ไปยัง Shoes

จงสร้างฟังก์ชันต่างๆ ในส่วนของ public ดังนี้
(ในกรณีที่ต้องจองพื้นที่ ให้ในศ.จองพื้นที่และคืนพื้นที่ให้เรียบร้อยด้วย ทุกๆ object จะต้องถูกสร้างและถูกทำลายได้อย่างถูกต้อง และน.ศ.สามารถสร้าง attributes และ function เพิ่มเติมได้ตามความเหมาะสม)

คลาส Food, Rice, InstantNoodles จงสร้าง

- default constructor ในการกำหนดค่าดังนี้
 - o Food ราคา 5 บาท
 - o Rice ข้าวหอมมะลิ ราคา 280 บาท
 - o InstantNoodles รสต้มยำกุ้ง ราคา 5 บาท
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอททริบิวต์แต่ละตัวโดยรับค่าจากพารามิเตอร์ โดยที่
 - o price และ weight ต้องมากกว่าศูนย์
 - o ข้าวมี 5 ชนิด
 - o flavor มี 4 รส
- ฟังก์ชัน get ในการรีเทิร์นค่าของแอททริบิวต์แต่ละตัว
- ฟังก์ชัน show โดยให้แสดงค่าทุกค่ารวมทั้งค่าใน superclass โดยให้สามารถใช้หลักการ dynamic binding ได้ด้วย

คลาส Shoes, Sneakers, HighHeels, SneakersHighHeels จงสร้าง

- default constructor ในการกำหนดค่าดังนี้
 - o Shoes ขนาด 37 ราคา 1000 บาท
 - o Sneakers แบบผูกเชือก ขนาด 35 ราคา 2800 บาท
 - o HighHeels ส้นหลังสูง 2 cm ขนาด 38 ราคา 2000 บาท
 - o SneakersHighHeels ส้นหน้าสูง 1.5 cm ส้นหลังสูง 2 cm แบบไม่ผูกเชือก ขนาด 36 ราคา 2500 บาท
- destructor ในการพิมพ์ค่าในคลาสของตนเองออกทางจอภาพ
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอททริบิวต์แต่ละตัวโดยรับค่าจากพารามิเตอร์ โดยที่ค่าของ double ต้องมีค่ามากกว่าศูนย์ และส้นหลังต้องมีค่ามากกว่าหรือเท่ากับส้นหน้าเสมอ
- ฟังก์ชัน get ในการรีเทิร์นค่าของแอททริบิวต์แต่ละตัว
- ฟังก์ชัน show โดยให้แสดงค่าทุกค่ารวมทั้งค่าใน superclass โดยให้สามารถใช้หลักการ dynamic binding ได้ด้วย
- operator double() โดยให้สามารถเรียกใช้ operator double() โดยใช้หลักการ dynamic binding ได้ด้วย แต่ละคลาสให้รีเทิร์นค่าดังนี้
 - o Shoes ให้รีเทิร์น size
 - o Sneakers ให้รีเทิร์น price
 - o HighHeels ให้รีเทิร์น height
 - o SneakersHighHeels ให้รีเทิร์น frontHeight
- แต่ละคลาสจงสร้างคอนสตรัคเตอร์ให้สามารถแปลงชนิดข้อมูลจาก double ไปเป็นอ็อบเจกต์ของคลาสตนเองได้ (convert จาก double ไปเป็น object ของคลาส) โดยให้กำหนดค่าต่างๆ ตามความเหมาะสม

คลาส SneakersHighHeels จงสร้าง

- จงสร้างฟังก์ชัน ในการกำหนดค่าให้กับทุกค่าโดยใช้หลักการ function overloading
- จงสร้าง operator ++ ในการเพิ่มความสูงของส้นหลัง ที่ละ 0.1 cm (ให้ทำเป็น friend function)
- ฟังก์ชันในการนับจำนวนอีอบเจกต์ของคลาส SneakersHighHeels ว่า ณ ปัจจุบันมีจำนวน อีอบเจกต์ ของ SneakersHighHeels อยู่ทั้งหมดเท่าไร ถึงแม้ว่าจะไม่มีอีอบเจกต์ของ SneakersHighHeels อยู่เลยก็ต้อง สามารถบอกได้ด้วย

คลาส Furniture, Cupboard, Pantry, ShoeCabinet จงสร้าง

- default constructor ในการกำหนดค่าให้กับแอทริบิวต์เป็นค่าเท่าใดก็ได้ โดยมีเงื่อนไขเดียวกันกับฟังก์ชัน set
- constructor กำหนดค่าแอทริบิวต์ทุกตัวรวมทั้ง base class โดยมีเงื่อนไขเดียวกันกับฟังก์ชัน set โดยที่
 - o กรณี food ให้จองพื้นที่ตามค่า numFood โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน food
 - o กรณี shoes ให้จองพื้นที่ตามค่า numShoes โดยยังไม่ต้องกำหนดค่าให้กับสมาชิกใน shoes
- destructor ในการคืนพื้นที่ และแสดงค่าในคลาสของตนเองออกทางจอภาพ
- ฟังก์ชัน set ในการกำหนดค่าให้กับแอทริบิวต์แต่ละตัวโดยรับค่าจากพารามิเตอร์ โดยที่
 - o price ต้องมากกว่าศูนย์
 - o color มีแค่ 3 สีตามตาราง
 - o numFood และ numShoes ต้องมากกว่าหรือเท่ากับศูนย์
 - o ถ้าค่า numFood เปลี่ยนจะต้องจองพื้นที่ใหม่ให้ food ด้วย
 - o ถ้าค่า numShoes เปลี่ยนจะต้องจองพื้นที่ใหม่ให้ shoes ด้วย
 - o ให้กำหนดค่าให้สมาชิกที่อยู่ใน food ที่ละตัว โดยระบุตำแหน่งที่เก็บในอาร์เรย์ เช่น setFood(int, Food*) เป็นต้น
 - o ให้กำหนดค่าให้สมาชิกที่อยู่ใน shoes ที่ละตัว โดยระบุตำแหน่งที่เก็บในอาร์เรย์ เช่น setShoes(int, Shoes*) เป็นต้น
- ฟังก์ชัน get ในการรีเทิร์นค่าของแอทริบิวต์แต่ละตัว กรณี food และ shoes ให้รีเทิร์นค่าสมาชิกในอาร์เรย์ที่ละ 1 ค่าโดยระบุตำแหน่งที่เก็บในอาร์เรย์
- ฟังก์ชัน show โดยให้แสดงค่าทุกค่ารวมทั้งค่าใน superclass โดยให้สามารถใช้หลักการ dynamic binding ได้ด้วย (กำหนดให้ show ของ Furniture และ Cupboard เป็น pure virtual function)

คลาส Pantry จงสร้าง

- Copy constructor ในการก๊อปปี้ค่าทุกค่าในคลาสของตนเองและbase class อย่างถูกต้อง
- operator << ในการพิมพ์ค่าทุกค่า รวมทั้งbase class

ใน main

- จงสร้างอ็อบเจกต์ดังนี้

ชื่ออ็อบเจกต์	คลาส	รายละเอียด
jasmineRice	Rice	ข้าวหอมมะลิ ขนาด 5 kg ราคา 320 บาท
munpuRice	Rice	ข้าวมันปู ขนาด 1 kg ราคา 90 บาท
tomyamkung	InstantNoodles	รสต้มยำกุ้ง ราคา 5 บาท
musub	InstantNoodles	รสหมูสับ ราคา 6 บาท
mala	InstantNoodles	รสหมาล่า ราคา 10 บาท
sneakers	Sneakers	แบบไม่ผูกเชือก ขนาด 38 ราคา 2500 บาท
highheels	HighHeels	ส้นหลังสูง 2 cm ขนาด 36 ราคา 1350 บาท
sneakersHighheels	SneakersHighHeels	แบบผูกเชือก ส้นหลังสูง 2 cm ส้นหน้าสูง 1.5 cm ขนาด 37 ราคา 1560 บาท
pantry1	Pantry	เป็นตู้สีขาว ราคา 14500 บาท ในตู้มี jasmineRice, tomyamkung, musub
pantry2	Pantry	เป็นตู้สีน้ำตาล ราคา 9900 บาท ในตู้มี munpuRice, mala
shoeCabinet	ShoeCabinet	เป็นตู้สีดำ ราคา 5600 บาท ในตู้มี sneakers, highheels sneakersHighheels

- จงสร้าง myFur เป็นอาร์เรย์ขนาด 3 เก็บพอยเตอร์ที่ชี้ไปยัง Furniture โดยให้สมาชิกแต่ละตัวชี้ไปยัง pantry1, pantry2, shoeCabinet ตามลำดับ
- ให้สมาชิกใน myFur เรียกใช้ฟังก์ชัน show โดยใช้หลักการ dynamic binding
- ใน shoeCabinet จงแปลงรองเท้าทั้ง 3 ให้เป็น double โดยใช้หลักการ dynamic binding และแสดงค่า double นั้นออกทางจอภาพ
- จงสร้าง a, b, c เป็นตัวแปรชนิด double โดยกำหนดค่าให้เป็น 1200.0, 2.5, 2.0 ตามลำดับ และจงแปลงตัวแปรเหล่านี้ให้เป็นอ็อบเจกต์ของคลาส Sneakers, Highheels, sneakersHighheels ตามลำดับ โดยให้ตั้งชื่ออ็อบเจกต์ที่สร้างขึ้นใหม่นี้ว่า aa, bb, cc ตามลำดับ จากนั้นให้แสดงค่า aa, bb, cc โดยเรียกใช้ฟังก์ชัน show()
- จงนำ aa, bb, cc ใส่เพิ่มเข้าไปในตู้ shoeCabinet และแสดงค่าที่เก็บใน shoeCabinet ทั้งหมดออกทางจอภาพ
- จงให้ pantry1, pantry2 แสดงค่าของตนเองโดยเรียกใช้ operator <<
- จงสร้าง pantry3 โดยให้เรียกใช้ copy constructor ในการก๊อปปี้ค่ามาจาก pantry2 และแสดงผลลัพธ์ของ pantry3 ออกทางจอภาพ
- จงคำนวณผลรวมของราคารองเท้าทุกคู่ในตู้ shoeCabinet และแสดงผลรวมออกทางจอภาพ
- จงตรวจสอบว่า ณ ขณะนี้ ถ้ามีอ็อบเจกต์ของคลาส SneakersHighHeels มากกว่า 0 ตัว ให้พิมพ์ว่า "hello my shoes" ถ้าไม่มีเลยให้พิมพ์ว่า "bye"

ឆេត

```
#include<iostream>
using namespace std;
class Food {
    double price;
public:
    Food() {
        price=5;
    }
    virtual ~Food() {}
    void setPrice(double p) {
        if (p>0)
            price = p;
        else
            price = 1;
    }
    double getPrice() {
        return price;
    }
    virtual void show() {
        cout<<"price: "<<price<<endl;
    }
};

class Rice: public Food {
    int type; //1.hommali 2.riceberry 3.klong 4.munpu 5.sangyod
    int weight;
public:
    Rice() {
        type=1; weight=1;
        setPrice(280);
    }
    void setType(int i) {
        if (i>=1 && i<=5)
            type=i;
        else
            type=1;
    }
    void setWeight(int w) {
        if (w>0)
            weight = w;
        else
            weight = 1;
    }
}
```

```

        int getType() {
            return type;
        }
        int getWeight() {
            return weight;
        }
        void show() {
            Food::show();
            cout<<"Type: "<<type<<endl;
            cout<<"Weight: "<<weight<<endl;
        }
};

class InstantNoodles: public Food {
    int flavor; //1.tomyum 2.musub 3.mumanao 4.mala
public:
    InstantNoodles() {
        flavor=1;
        setPrice(5);
    }
    void setFlavor(int f) {
        if (f>=1 && f<=4)
            flavor=f;
        else
            flavor=1;
    }
    int getFlavor() {
        return flavor;
    }
    void show() {
        Food::show();
        cout<<"Flavor: "<<flavor<<endl;
    }
};

```

```

class Shoes {
    double price;
    double size;
public:
    Shoes() {
        price=1000;
        size=37;
    }
}

```

```

    shoes(double d) {
        setPrice(d);
        setSize(36);
    }
    virtual ~Shoes() {
        showShoes();
    }
    void showShoes() {
        cout<<"Shoes price: "<<price<<endl;
        cout<<"Shoes size: "<<size<<endl;
    }
    void setPrice(double p) {
        if (p>0)
            price=p;
        else
            price=1;
    }
    void setSize(double s) {
        if (s>0)
            size=s;
        else
            size=35;
    }
    double getPrice() {
        return price;
    }
    double getSize() {
        return size;
    }
    virtual void show() {
        showShoes();
    }
    virtual operator double() {
        return size;
    }
};

class Sneakers: virtual public Shoes {
    bool shoelace;
public:
    Sneakers() {
        shoelace=true;
        setPrice(2800);
        setSize(35);
    }
}

```

```

    Sneakers(double d) {
        setPrice(d);
        setSize(36);
        setShoelace(true);
    }
    ~Sneakers() {
        showSneakers();
    }
    void showSneakers() {
        cout<<"Sneakers shoelace: "<<shoelace<<endl;
    }
    void setShoelace(bool b) {
        shoelace = b;
    }
    bool getShoelace() {
        return shoelace;
    }
    void show() {
        Shoes::show();
        showSneakers();
    }
    operator double() {
        return getPrice();
    }
};

```

```

class HighHeels: virtual public Shoes {
    double height;
public:
    HighHeels() {
        height=2;
        setPrice(2000);
        setSize(38);
    }
    HighHeels(double d) {
        setHeight(d);
        setPrice(2000);
        setSize(38);
    }
    ~HighHeels() {
        showHighHeels();
    }
}

```

```

void showHighHeels() {
    cout<<"HighHeels height: "<<height<<endl;
}
void setHeight(double h) {
    if (h>0)
        height=h;
    else
        height=1;
}
double getHeight() {
    return height;
}
void show() {
    Shoes::show();
    showHighHeels();
}
operator double() {
    return height;
}
};

class SneakersHighHeels: public Sneakers, public HighHeels {
    double frontHeight;
    static int num;
public:
    SneakersHighHeels() {
        frontHeight=1.5;
        setPrice(2500);
        setHeight(2);
        setShoelace(false);
        setSize(36);
        num++;
    }
    SneakersHighHeels(double d) {
        setFrontHeight(d);
        setPrice(2500);
        setHeight(2);
        setShoelace(false);
        setSize(36);
        num++;
    }
    ~SneakersHighHeels() {
        showSneakersHighHeels();
        num--;
    }
}

```

```

void showSneakersHighHeels() {
    cout<<"SneakersHighHeels frontHeight: "
        <<frontHeight<<endl;
}
void setFrontHeight(double d) {
    if (d>0 && getHeight()>=frontHeight)
        frontHeight=d;
    else
        frontHeight=getHeight();
}
double getFrontHeight() {
    return frontHeight;
}
void show() {
    showShoes();
    showSneakers();
    showHighHeels();
    showSneakersHighHeels();
}
operator double() {
    return frontHeight;
}
void set(double f) {
    setFrontHeight(f);
}
void set(double f, double h) {
    setHeight(h);
    setFrontHeight(f);
}
friend SneakersHighHeels operator++(SneakersHighHeels &s) {
    s.setHeight(s.getHeight()+0.1);
    return s;
}
static int count() {
    return num;
}
};
int SneakersHighHeels::num;
class Furniture {
    double price;
public:
    Furniture() {
        price=100;
    }
}

```

```

Furniture(double p) {
    setPrice(p);
}
virtual ~Furniture() {
    showFurniture();
}
void showFurniture() {
    cout<<"Furniture price: "<<price<<endl;
}
void setPrice(double p) {
    if (p>0)
        price=p;
    else
        price=100;
}
double getPrice() {
    return price;
}
virtual void show() = 0;
};

```

```

class Cupboard: public Furniture {
    int color; //1.white 2.black 3.brown
public:
    Cupboard() {
        setColor(1);
    }
    Cupboard(int c, double p) {
        setColor(c);
        setPrice(p);
    }
    ~Cupboard() {
        showCupboard();
    }
    void setColor(int c) {
        if (c>=1 && c<=3)
            color = c;
        else
            color = 1;
    }
    int getColor() {
        return color;
    }
}

```



```

        }
        else {
            delete[] food;
            numFood = n;
            food = new Food*[numFood];
        }
    }
    void setFood(int i, Food *f) {
        food[i]=f;
    }
    int getNumFood() {
        return numFood;
    }
    Food* getFood(int i) {
        return food[i];
    }
    void show() {
        showFurniture();
        showCupboard();
        showPantry();
    }
};

ostream &operator<<(ostream &o, Pantry &p) {
    p.show();
    return o;
}

class ShoeCabinet: public Cupboard {
    int numShoes;
    Shoes **shoes;
public:
    ShoeCabinet() {
        numShoes=0;
        shoes=0;
    }
    ShoeCabinet(int n, int c, double p): Cupboard(c, p) {
        numShoes=0;
        shoes=0;
        setNum(n);
    }
    ~ShoeCabinet() {
        delete[] shoes;
    }
}

```

```

void showShoeCabinet() {
    cout<<"ShoeCabinet numShoes: "<<numShoes<<endl;
    for (int i=0; i<numShoes; i++)
        shoes[i]->show();
}
void setNum(int n) {
    if (n == numShoes)
        return;
    if (n<=0) {
        numShoes=0;
        delete[] shoes;
        shoes=0;
    }
    else {
        delete[] shoes;
        numShoes = n;
        shoes = new Shoes*[numShoes];
    }
}
void setShoes(int i, Shoes *f) {
    shoes[i]=f;
}
int getNumShoes() {
    return numShoes;
}
Shoes* getShoes(int i) {
    return shoes[i];
}
void show() {
    showFurniture();
    showCupboard();
    showShoeCabinet();
}

};

int main() {
    Rice jasmineRice;
    jasmineRice.setType(1);
    jasmineRice.setPrice(320);
    jasmineRice.setWeight(5);

    Rice munpuRice;
    munpuRice.setType(4);
    munpuRice.setPrice(90);

```

```
munpuRice.setWeight(1);
```

```
InstantNoodles tomyamkung;  
tomyamkung.setFlavor(1);  
tomyamkung.setPrice(5);
```

```
InstantNoodles musub;  
musub.setFlavor(2);  
musub.setPrice(6);
```

```
InstantNoodles mala;  
mala.setFlavor(4);  
mala.setPrice(10);
```

```
Sneakers sneakers;  
sneakers.setShoelace(false);  
sneakers.setSize(38);  
sneakers.setPrice(2500);
```

```
HighHeels highheels;  
highheels.setHeight(2);  
highheels.setSize(36);  
highheels.setPrice(1350);
```

```
SneakersHighHeels sneakersHighheels;  
sneakersHighheels.setHeight(2);  
sneakersHighheels.setFrontHeight(1.5);  
sneakersHighheels.setShoelace(true);  
sneakersHighheels.setSize(37);  
sneakersHighheels.setPrice(1560);
```

```
Pantry pantry1(3, 1, 14500);  
pantry1.setFood(0, &jasmineRice);  
pantry1.setFood(1, &tomyamkung);  
pantry1.setFood(2, &musub);
```

```
Pantry pantry2(2, 3, 9900);  
pantry2.setFood(0, &munpuRice);  
pantry2.setFood(1, &mala);
```

```
ShoeCabinet shoeCabinet(3, 2, 5600);  
shoeCabinet.setShoes(0, &sneakers);  
shoeCabinet.setShoes(1, &highheels);  
shoeCabinet.setShoes(2, &sneakersHighheels);
```

```
Furniture *myFur[3] = {&pantry1, &pantry2, &shoeCabinet};
```

```
for (int i=0; i<3; i++) {
    myFur[i]->show();
    cout<<endl<<endl;
}
```

```
cout<<".....Convert to Double....."<<endl;
for (int i=0; i<shoeCabinet.getNumShoes(); i++)
    cout<<(double)(*shoeCabinet.getShoes(i))<<endl;
```

```
cout<<"\n.....Convert from Double to Object ....."<<endl;
double a, b, c;
a=1200;
b=2.5;
c=2.0;
Sneakers aa = a;
HighHeels bb = b;
SneakersHighHeels cc = c;
aa.show();
bb.show();
cc.show();
```

```
cout<<"\n..... add shoes ....."<<endl;
shoeCabinet.setNum(6);
shoeCabinet.setShoes(0, &sneakers);
shoeCabinet.setShoes(1, &highheels);
shoeCabinet.setShoes(2, &sneakersHighheels);
shoeCabinet.setShoes(3, &aa);
shoeCabinet.setShoes(4, &bb);
shoeCabinet.setShoes(5, &cc);
shoeCabinet.show();
```

```
cout<<"\n.....cout pantry1 pantry2....."<<endl;
cout<<endl<<pantry1<<endl;
cout<<endl<<pantry2<<endl;
```

```
cout<<"\n.....pantry3....."<<endl;
Pantry pantry3(pantry2);
pantry3.show();
```

```
cout<<"\n.....compute price....."<<endl;
double total=0;
```

```
    for (int i=0; i<shoeCabinet.getNumShoes(); i++)
        total+=shoeCabinet.getShoes(i)->getPrice();
    cout<<"\ntotal = "<<total<<endl;

    cout<<"\n.....count....."<<endl;
    if (SneakersHighHeels::count()>0)
        cout<<"hello my shoes"<<endl;
    else
        cout<<"bye"<<endl;

    cout<<"\n... End Program ..."<<endl;
}
```